CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

Sección 1: Creación y Configuración de Cuenta en Render

Paso 1: Acceso al Portal

- Accede al sitio web oficial de Render. Render es una plataforma como servicio (PaaS) en la nube que automatiza el despliegue de aplicaciones, bases de datos y sitios estáticos sin necesidad de gestionar la infraestructura subyacente.
- Haz clic en el botón **Get Started** situado en la esquina superior derecha para iniciar el proceso de registro y creación de tu cuenta en la plataforma.

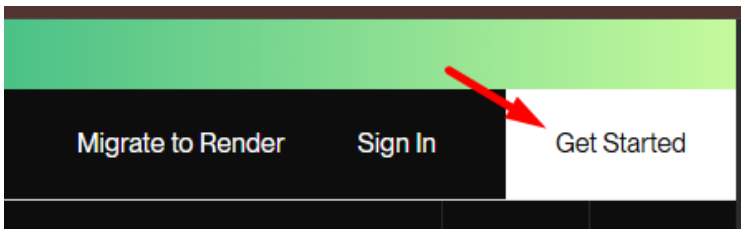


Figura 1: Botón Get Started en el portal de Render.

Paso 2: Registro e Integración con GitHub

- Selecciona la opción de autenticación mediante **GitHub** (Single Sign-On o SSO) para vincular directamente tu identidad de desarrollador. Esto automatiza la integración continua (CI/CD), permitiendo que Render detecte cambios en tus repositorios y re-despliegue el código de forma automática.
- Sigue las instrucciones en pantalla para autorizar los permisos de lectura e integración con tu cuenta de GitHub, permitiendo a Render conectarse de manera segura vía OAuth.

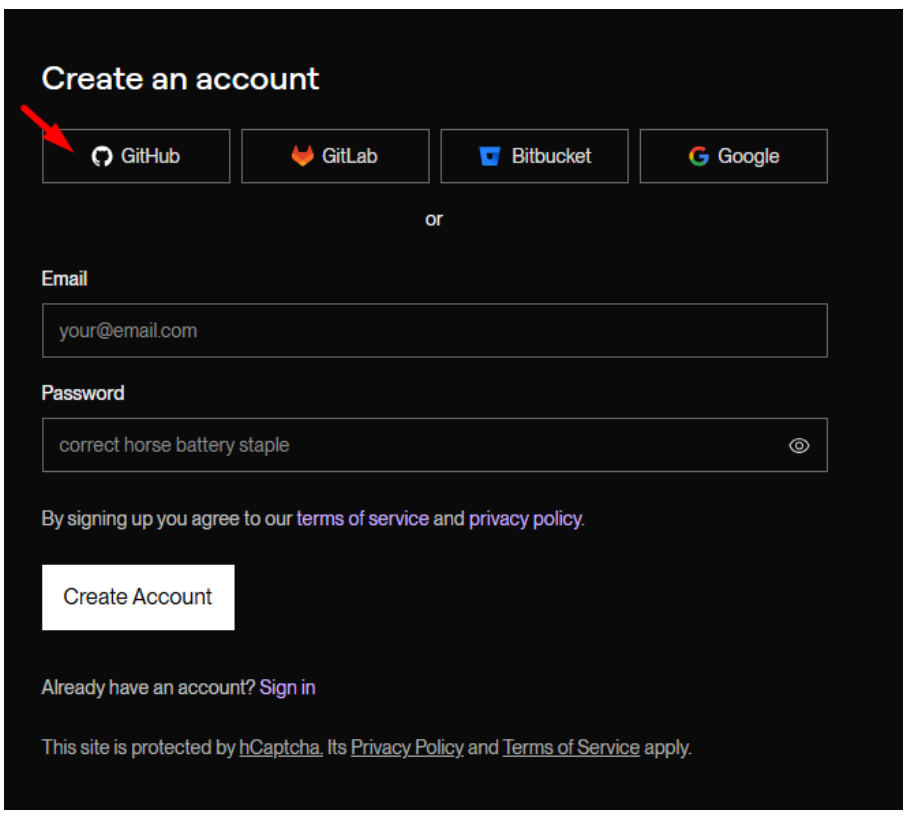


Figura 2: Opción de registro e inicio de sesión con GitHub en Render.

Paso 3: Configuración de Autenticación de Dos Factores (MFA / 2FA)

Para garantizar la seguridad de la cuenta y proteger el código fuente e infraestructura contra accesos no autorizados, se requiere la configuración obligatoria de autenticación multifactor (MFA / 2FA):

- Descarga una app de autenticación para MFA desde tu dispositivo móvil, por ejemplo **Google Authenticator**, **Microsoft Authenticator** o **Guardián**. Estas aplicaciones generan contraseñas de un solo uso basadas en el tiempo (TOTP).
- Escanea el código QR que genera y muestra GitHub inmediatamente después de dar clic en el botón verde de confirmación para vincular la clave secreta compartida.
- Ingresa en el campo correspondiente el código TOTP (token dinámico de 6 dígitos) que genera la app de autenticación para validar y activar la doble protección.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

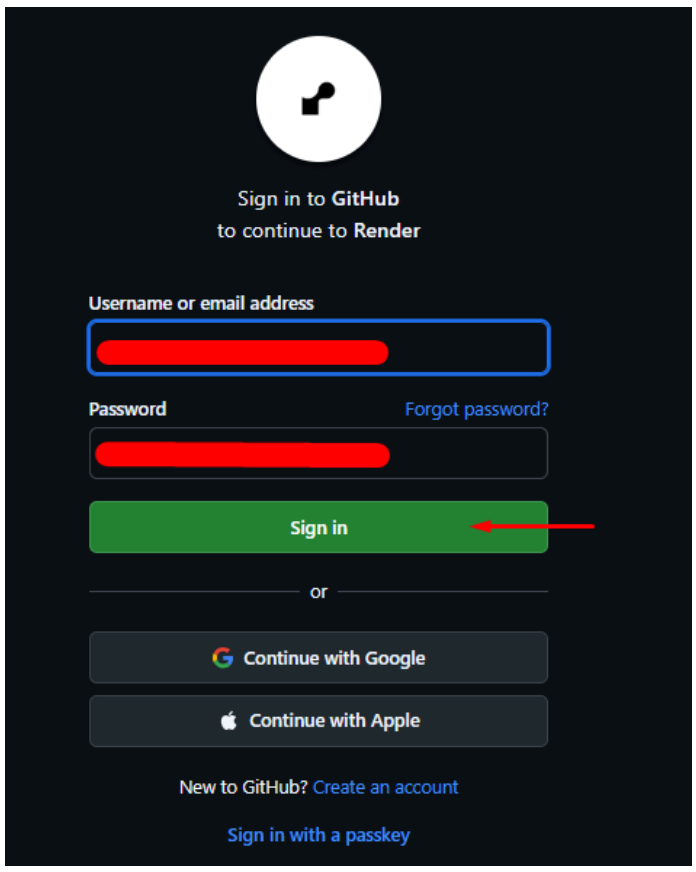


Figura 3: Pantalla de inicio de sesión en GitHub para continuar a Render.

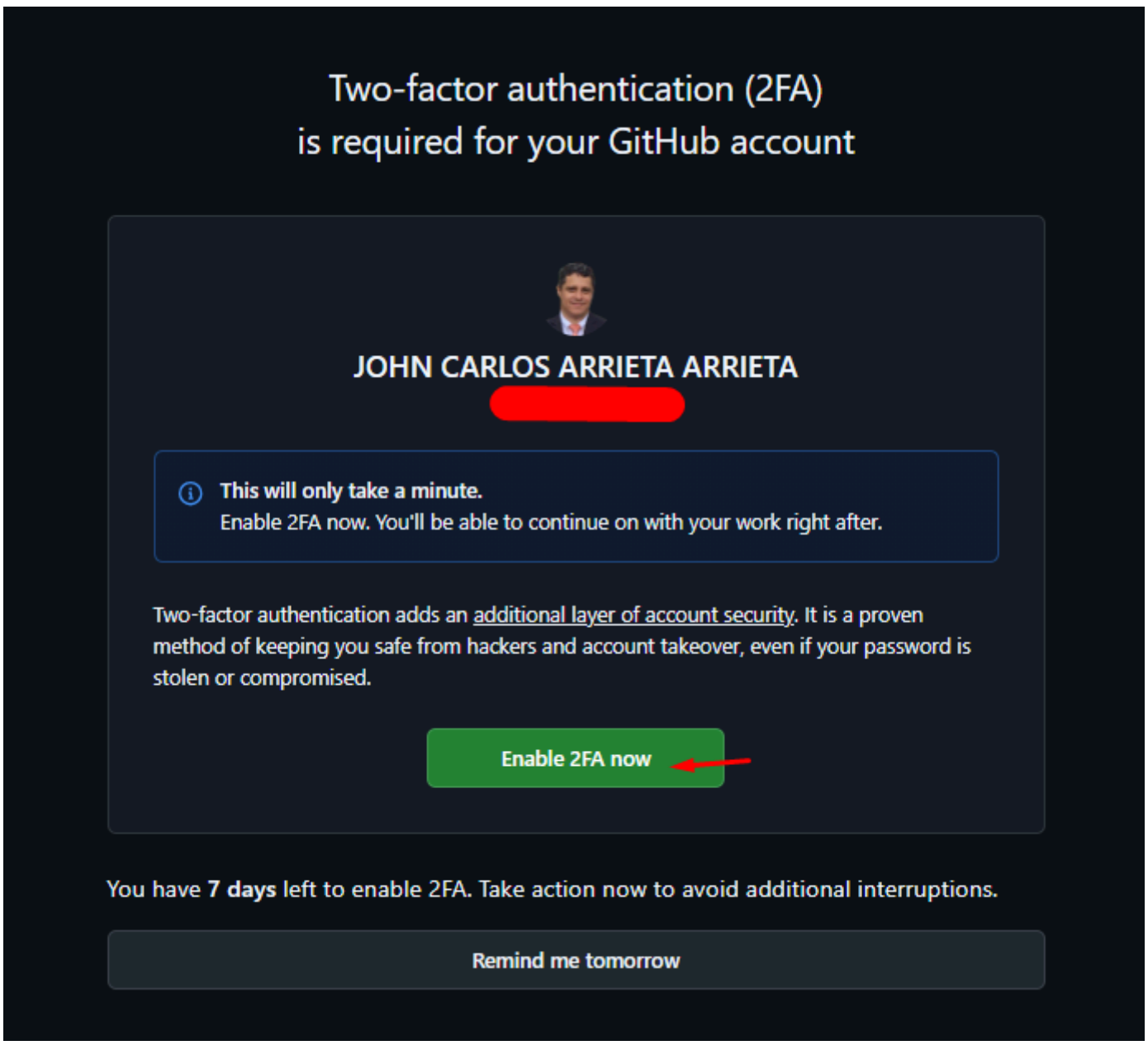


Figura 4: Solicitud de activación de la autenticación de dos factores (2FA) en GitHub.

1. Descargar una app de autenticación para MFA desde tu dispositivo móvil, por ejemplo Google Authenticator (la que use en esta guía) o Microsoft Authenticator o Guardián
2. Escanear el código QR que genera y muestra GH inmediatamente después de dar clic en el botón verde señalado en la imagen anterior.
3. Ingresar en el campo el código TOTP (token) que genera la APP de autentificación

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** cc + by + sa

---

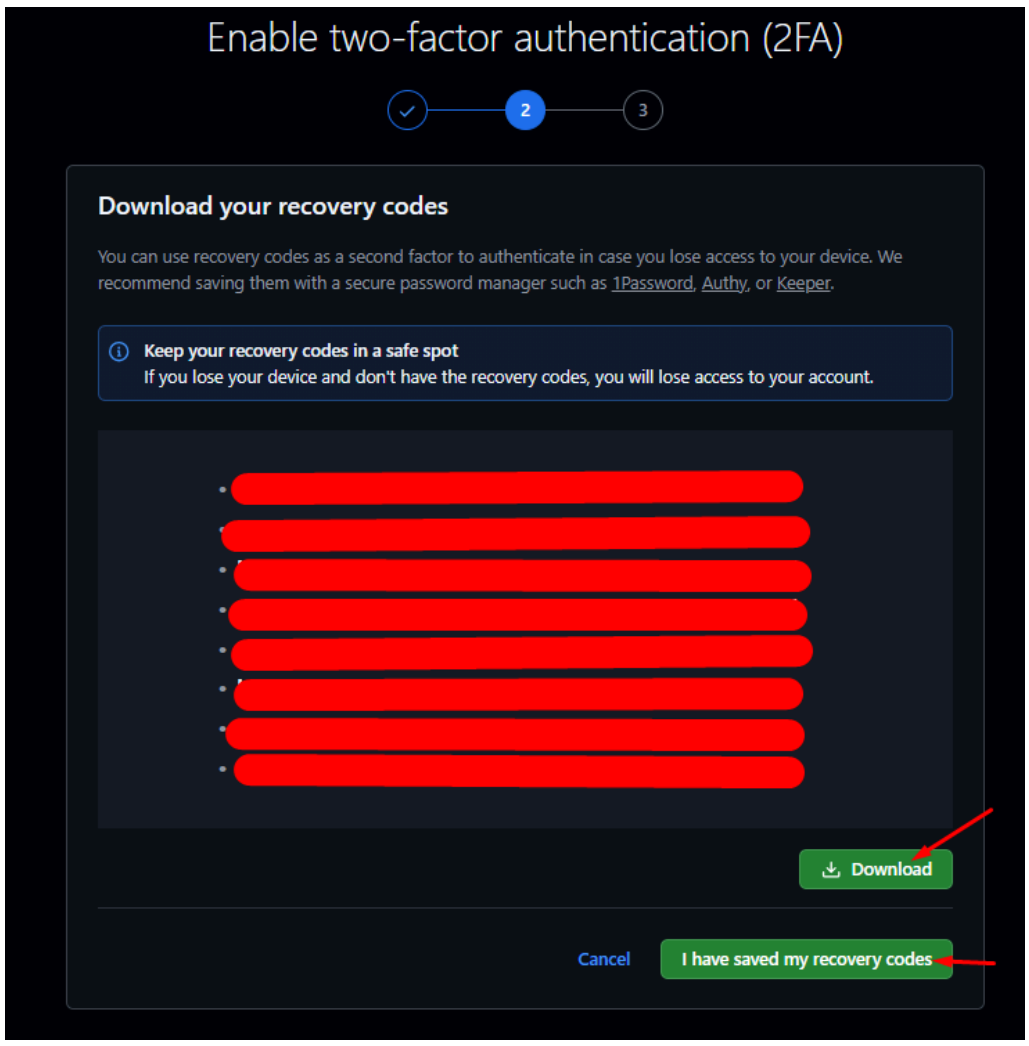


Figura 5: Pantalla para la descarga y confirmación de los códigos de recuperación de 2FA.

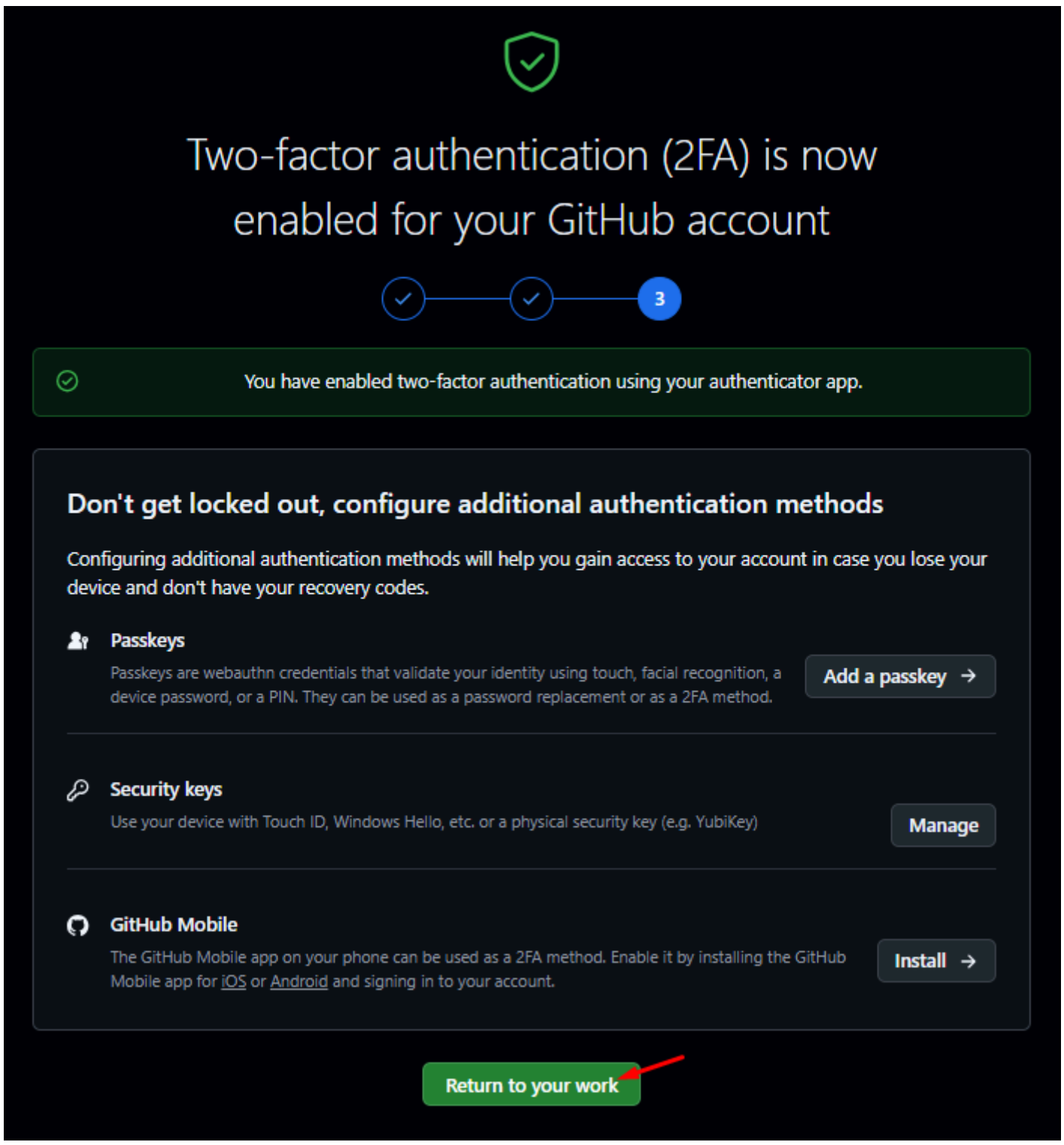


Figura 6: Confirmación de activación exitosa de 2FA en la cuenta de GitHub.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

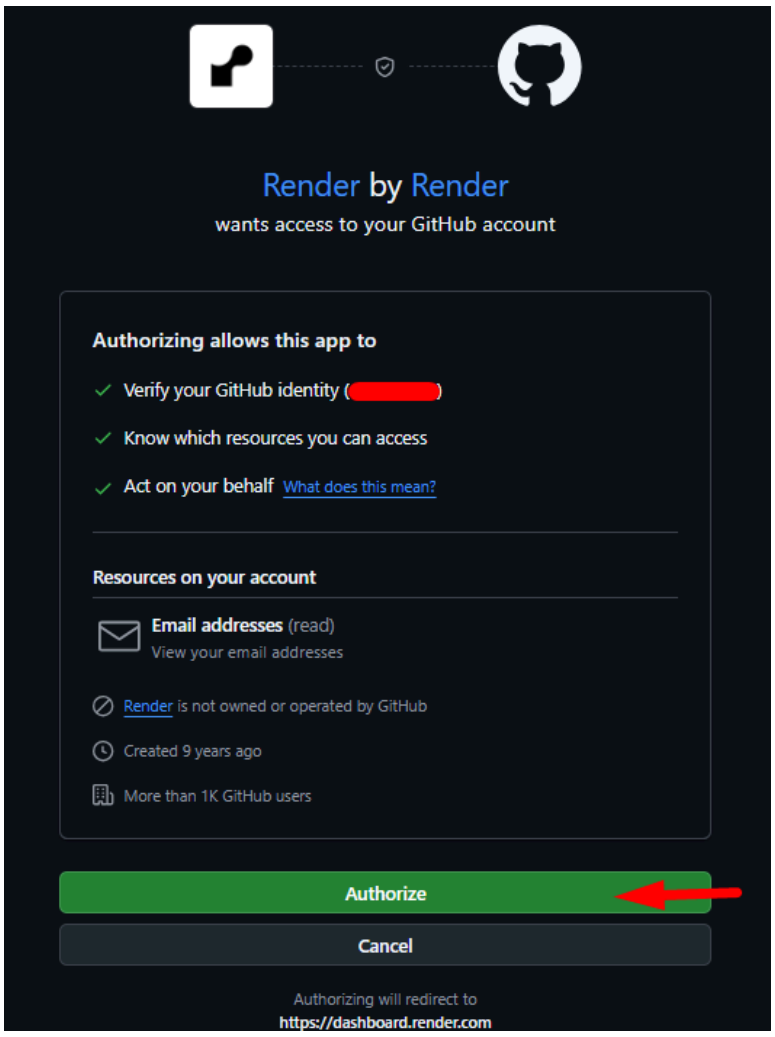


Figura 7: Autorización de la aplicación Render para acceder a la cuenta de GitHub.

Sección 2: Despliegue de Aplicación Spring Boot desde GitHub

Paso 4: Creación de un Nuevo Servicio Web

- Una vez dentro del panel principal (*Dashboard*) de Render, haz clic en el botón **New +** en la esquina superior derecha y selecciona **Web Service**.
- Esta modalidad permite alojar servicios backend persistentes como APIs REST en Java/Spring Boot, gestionando automáticamente el tráfico entrante, los certificados SSL/TLS (HTTPS) y el ciclo de vida del contenedor.

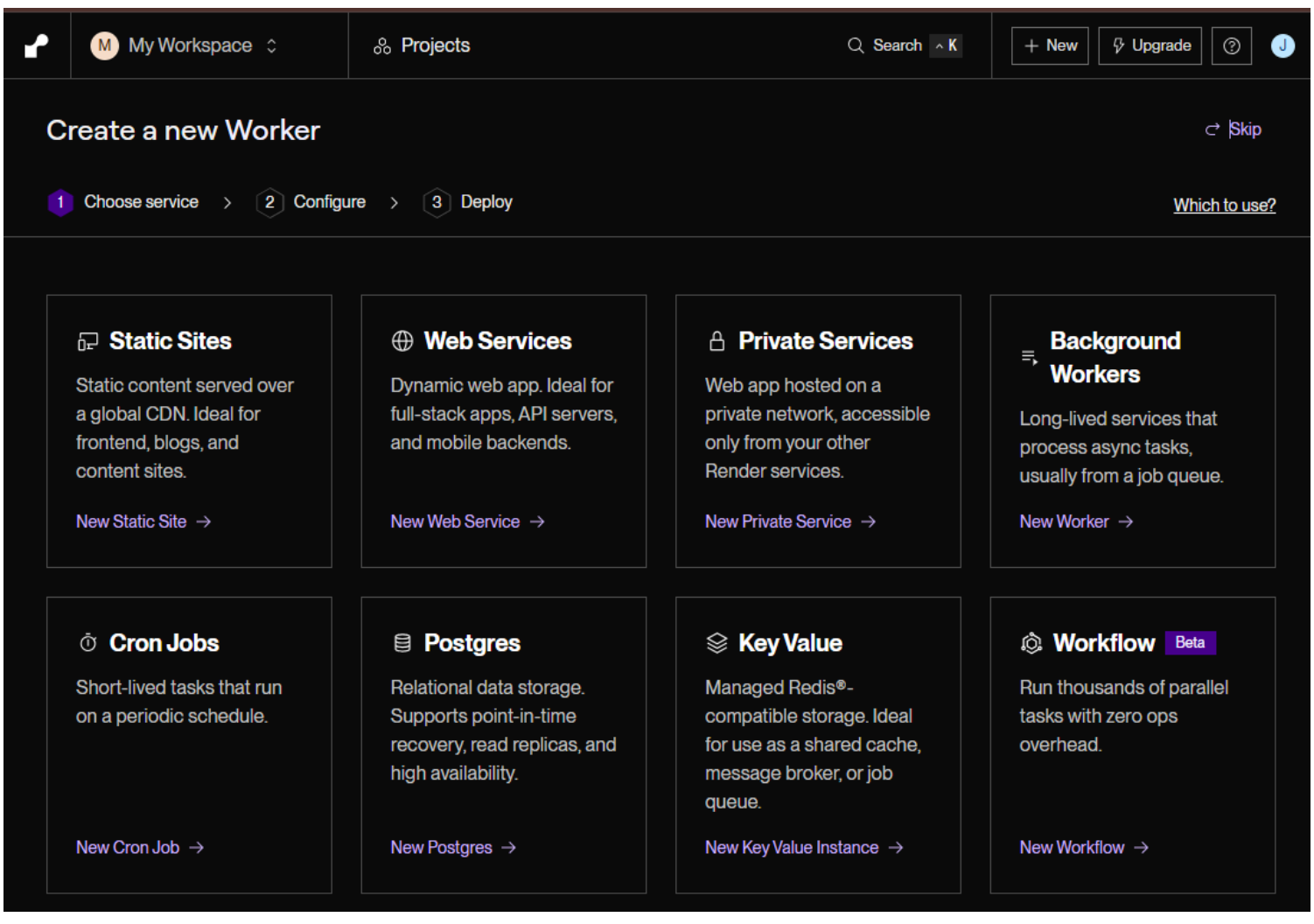


Figura 8: Opciones de creación de servicios en el panel de Render.

CC-BY.CC **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

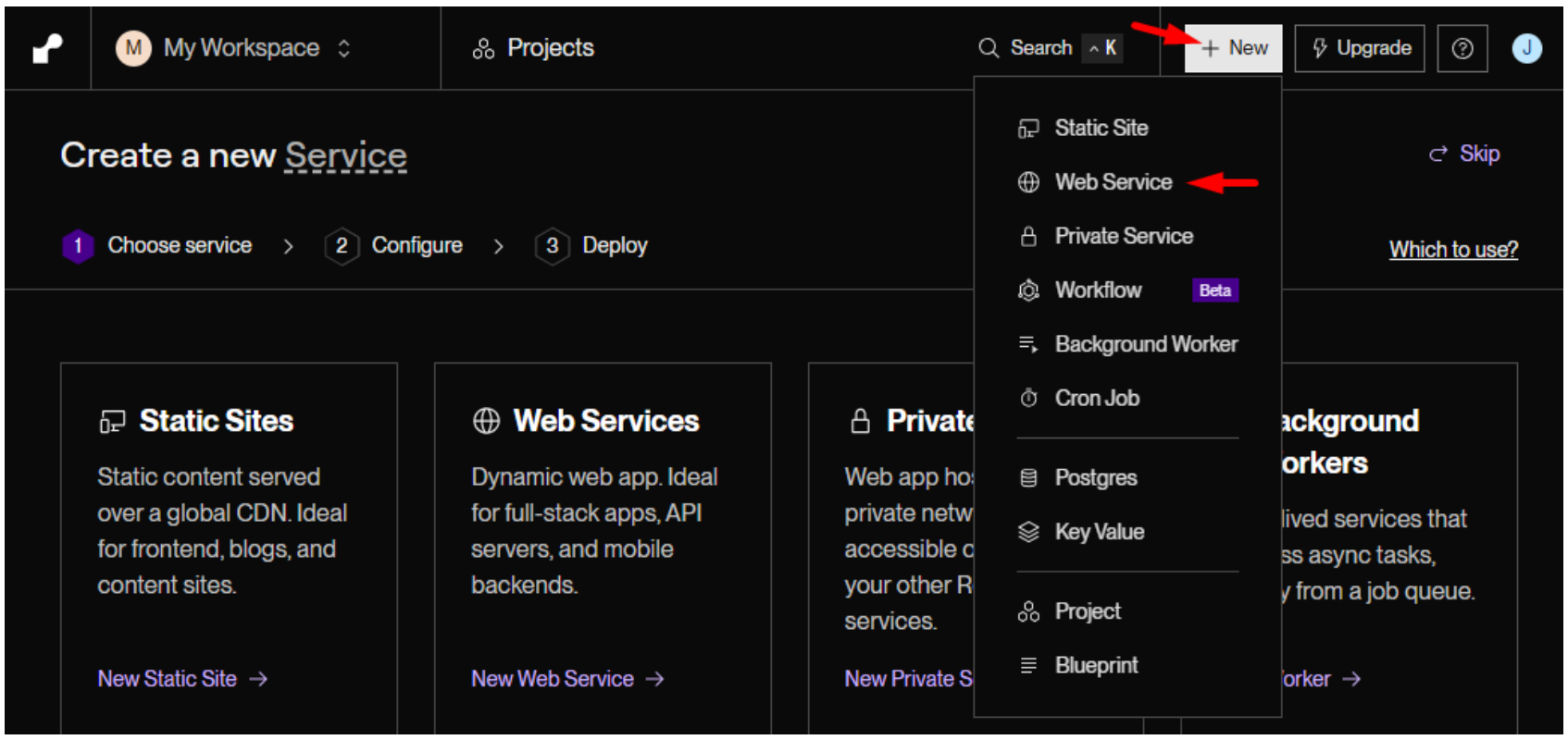


Figura 9: Menú desplegable para seleccionar la creación de un nuevo Web Service.

Paso 5: Vinculación del Repositorio de Código

- Busca en el listado de repositorios conectados de GitHub el proyecto Spring Boot que deseas desplegar y haz clic en **Connect**.
- *Nota:* Si el repositorio es privado, asegúrate de conceder permisos explícitos de acceso a Render en la configuración de la GitHub App para que pueda clonar el código durante el proceso de construcción.

Selección del proveedor Git en Render (Primera imagen)

- Muestra la pantalla **`New Web Service`** en Render.
- Dentro de la pestaña **`Git Provider`**, la plataforma indica que no hay repositorios vinculados aún (`No repositories found`).
- Una flecha roja señala el botón **`GitHub`**, el cual redirecciona hacia la autenticación oficial de GitHub para autorizar el acceso.

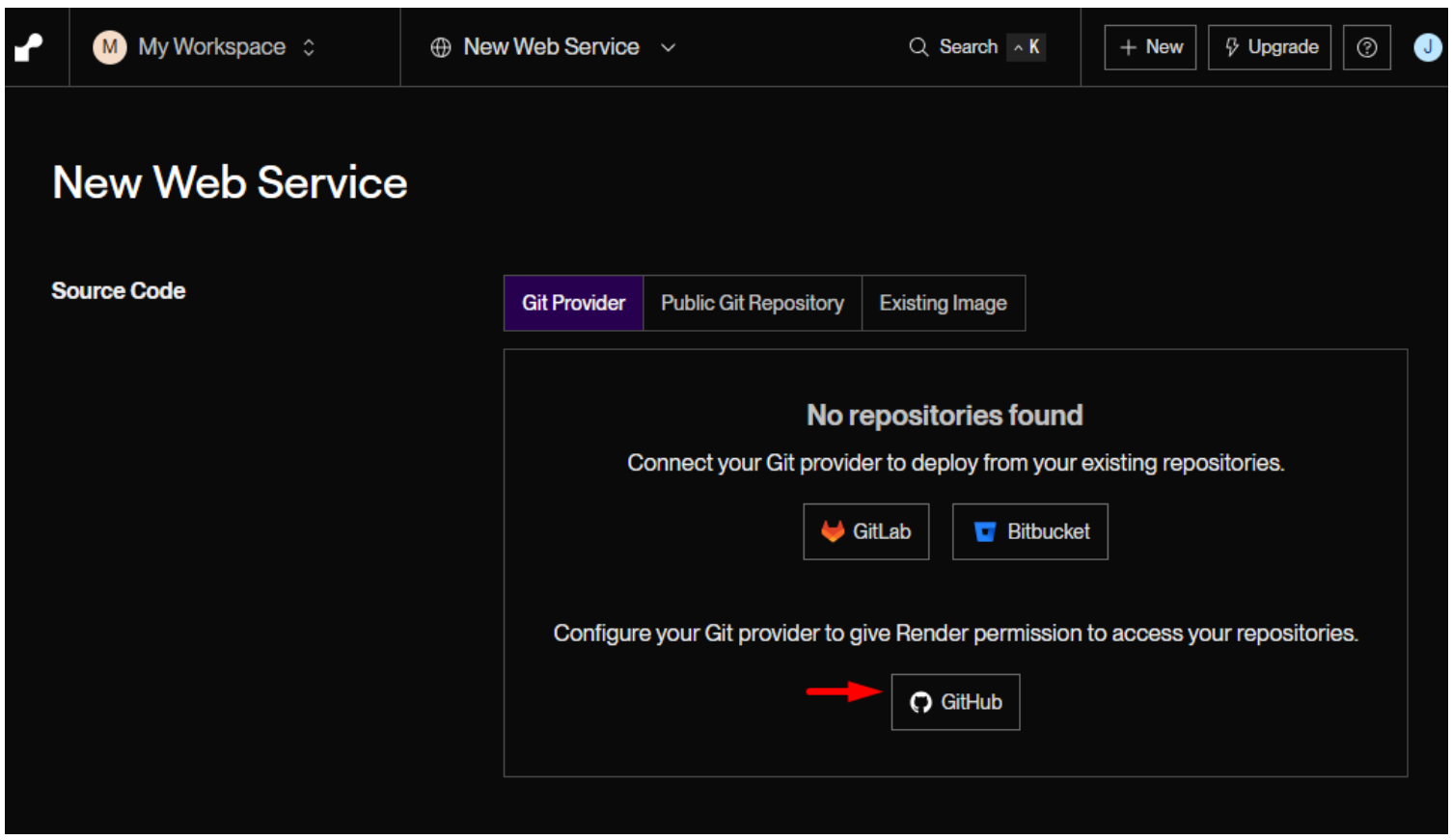


Figura 10: Conexión con el proveedor Git (GitHub) en Render.

Instalación de la aplicación de Render en GitHub (Segunda imagen)

- Es la pantalla de permisos **`Install Render`** dentro de GitHub para la cuenta de usuario especificada.
- La primera flecha señala la selección de **`All repositories`**, concediendo acceso a todos los repositorios actuales y futuros de la cuenta.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

- Se enlistan los permisos de lectura/escritura requeridos por Render (código, acciones, despliegues, etc.).
- La segunda flecha abajo indica el botón verde **`Install`** para autorizar la integración.

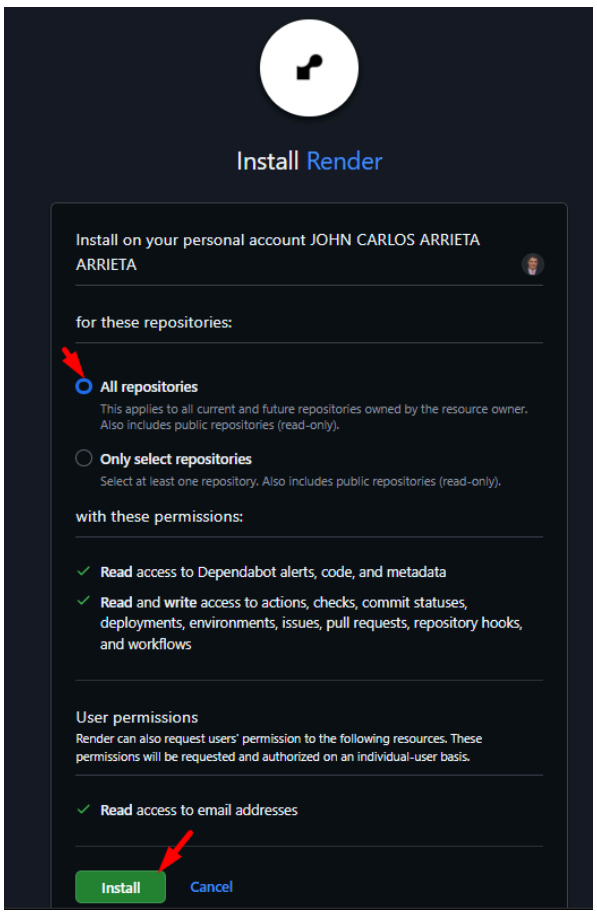


Figura 11: Configuración de permisos de acceso a repositorios para Render en GitHub.

Confirmación de identidad / Modo Sudo (Tercera imagen)

- Presenta la pantalla de seguridad de GitHub **`Confirm access`** requerida al instalar aplicaciones o realizar cambios sensibles (*sudo mode*).
- Muestra el usuario autenticado y dos flechas rojas: la primera apunta al campo de contraseña (o código de autenticación de dos factores) y la segunda señala el botón verde **`Verify`** para confirmar la acción.

En este paso debe aparecer el repositorio `users-management-spring-boot-hexagonal-ddd` al que le hicimos ***FORK*** en anteriormente y sobre el cual hemos trabajado durante toda esta guia didactica.

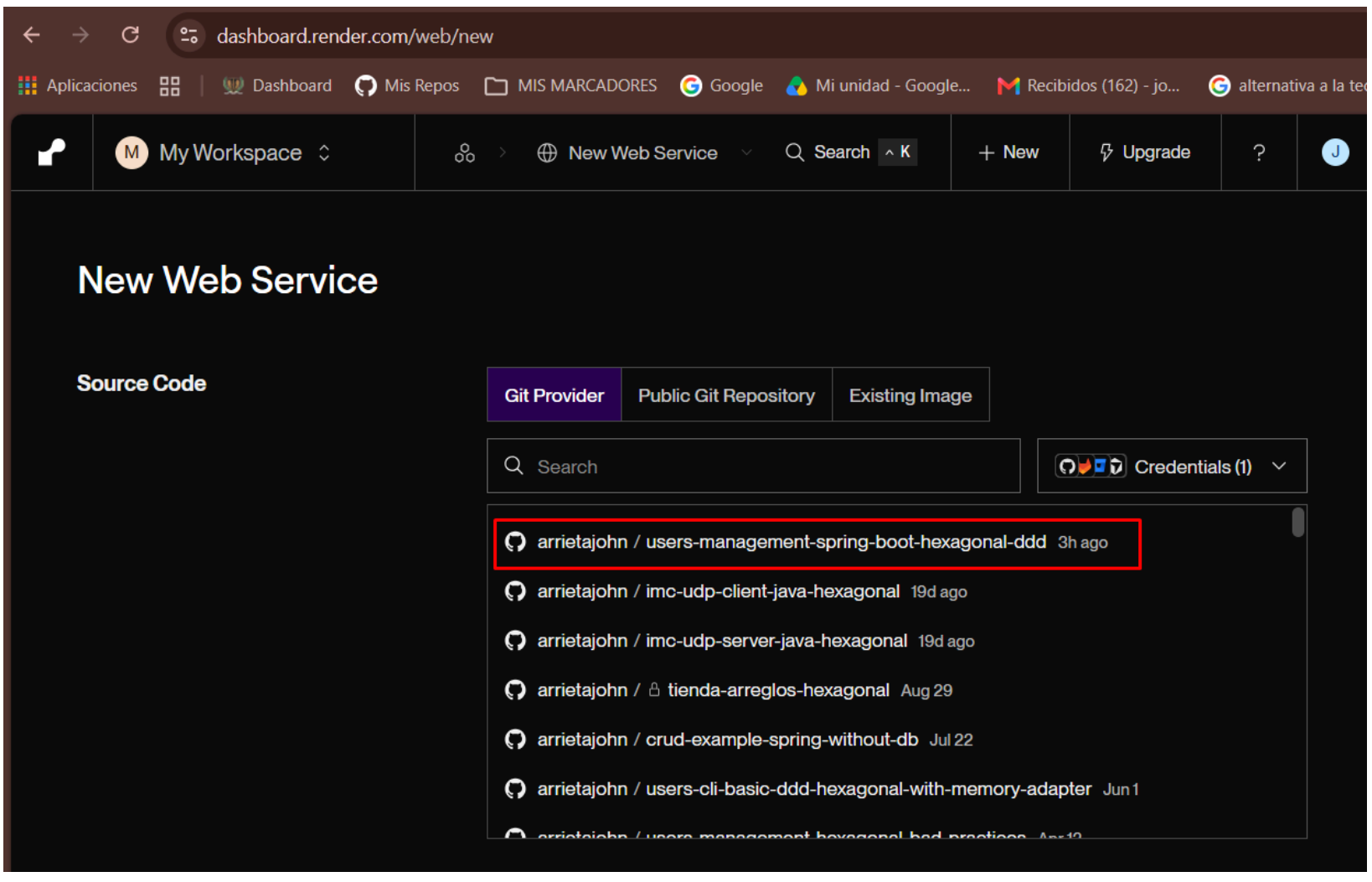


Figura 12: Selección del repositorio del proyecto en la lista de repositorios conectados.

CC-BY.CC **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

En la lista de repositorios sincronizados, localiza el proyecto `users-management-spring-boot-hexagonal-ddd` (el cual bifurcaste mediante un Fork en pasos anteriores) y selecciónalo para proceder con la vinculación.

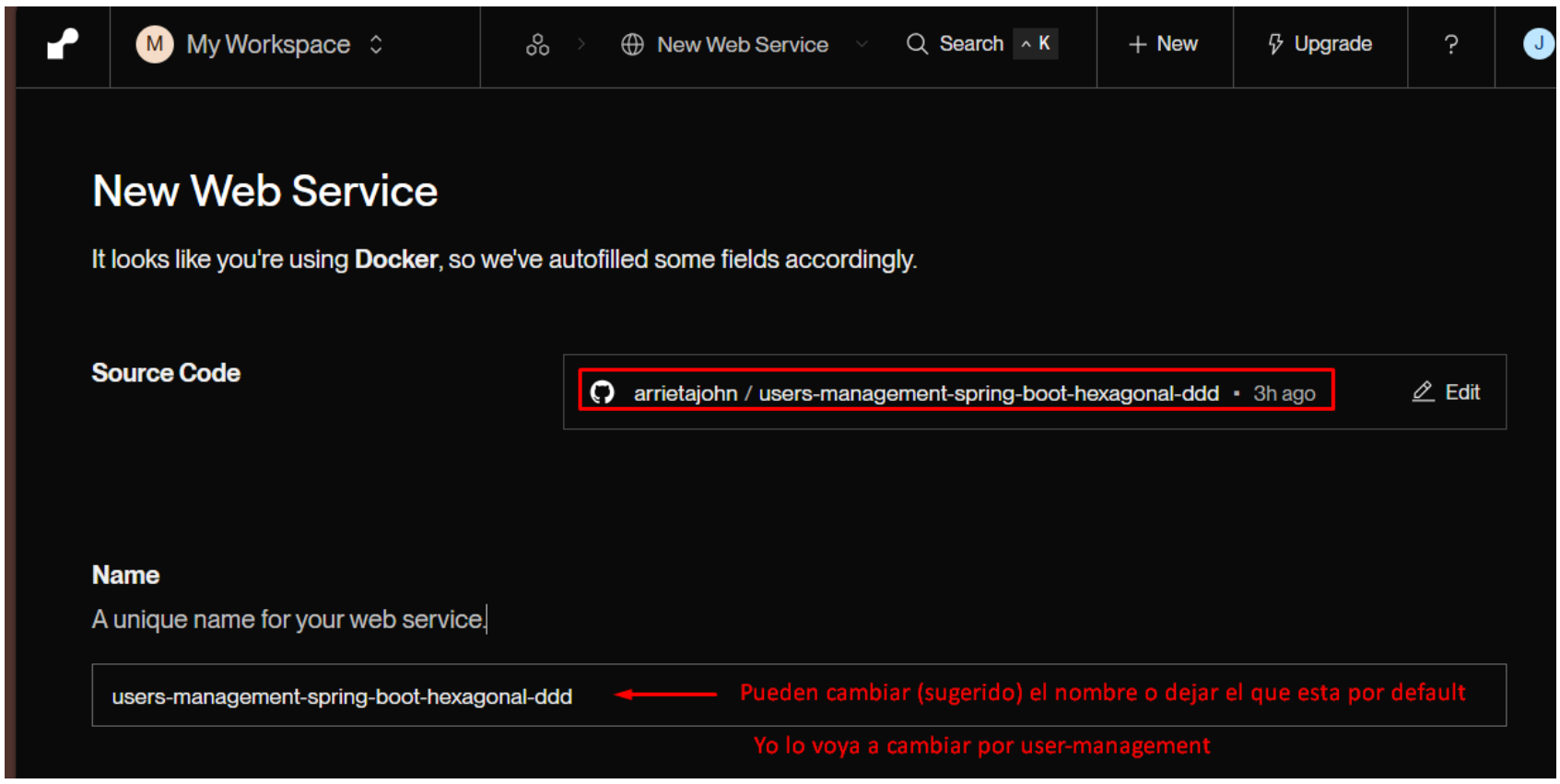


Figura 13: Repositorio vinculado y asignación del nombre para el servicio web.

Paso 6: Definición del Nombre del Servicio Web

Asigna un nombre único a tu Web Service en el campo **Name**. Puedes mantener el nombre por defecto o simplificarlo a algo más identificable, como por ejemplo `user-management`. Este nombre se utilizará para generar el subdominio público en Render (por ejemplo, `nombre-servicio.onrender.com`).

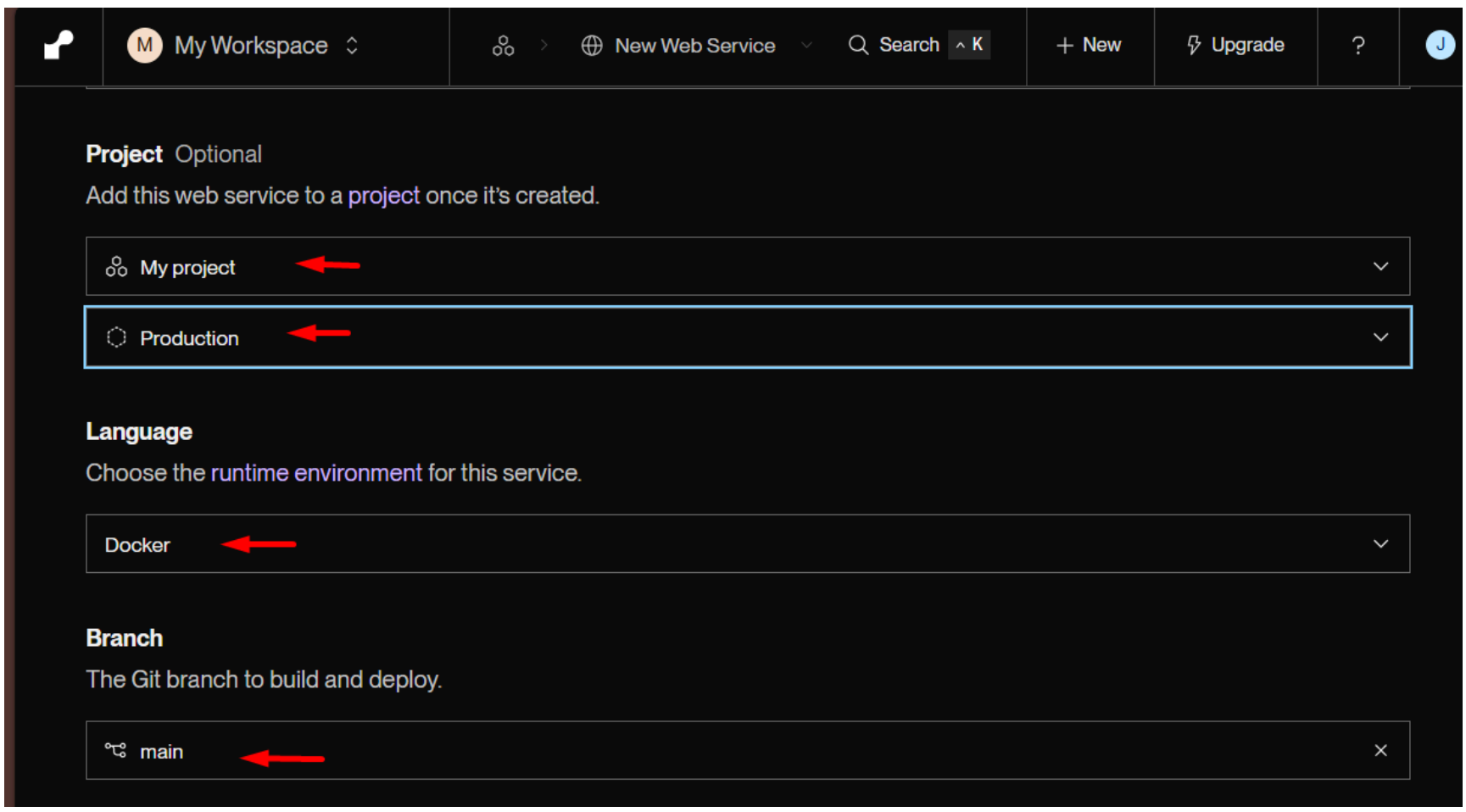


Figura 14: Configuración de proyecto, entorno de ejecución (Docker) y rama principal.

Paso 7: Configuración del Entorno de Ejecución (Runtime) y Rama

Selecciona el proyecto de destino (por ejemplo, **My project / Production**), asegúrate de que el entorno o Runtime seleccionado sea **Docker** (ya que la aplicación cuenta con un Dockerfile que empaqueta Java y las dependencias) y selecciona la rama principal de código (usualmente `main`). Esto garantiza construcciones reproducibles y aisladas.

CC-BY.CC **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

Figura 15: Selección de la región geográfica para el despliegue del servidor.

Paso 8: Selección de la Región Geográfica

Elige la región donde se desplegará el servidor (por ejemplo, **Virginia (US East)**). Es recomendable seleccionar la región físicamente más cercana a la base de datos o a los usuarios finales para minimizar el RTT (Round Trip Time) y la latencia de red en las peticiones HTTP.

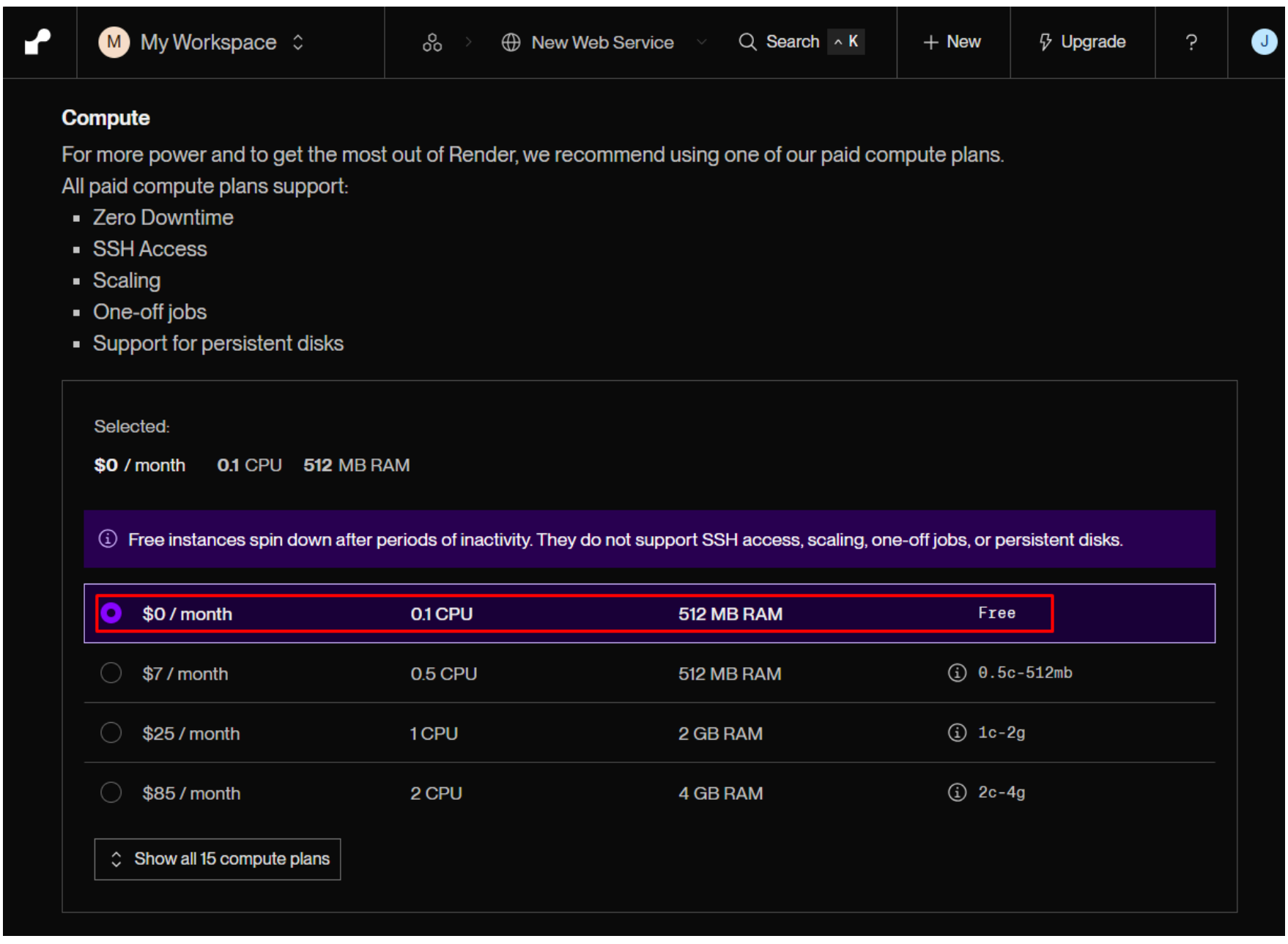


Figura 16: Selección del plan de cómputo gratuito (Free).

Paso 9: Selección del Plan de Cómputo

Selecciona la opción gratuita **Free ($0/month)** que ofrece 0.1 CPU y 512 MB de Memoria RAM. Ten en cuenta que las instancias gratuitas entran en reposo (sleep mode) tras 15 minutos de inactividad para optimizar recursos, sufriendo un tiempo de arranque en frío (cold start) de unos segundos al recibir una nueva petición.

CC-BY.CC **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

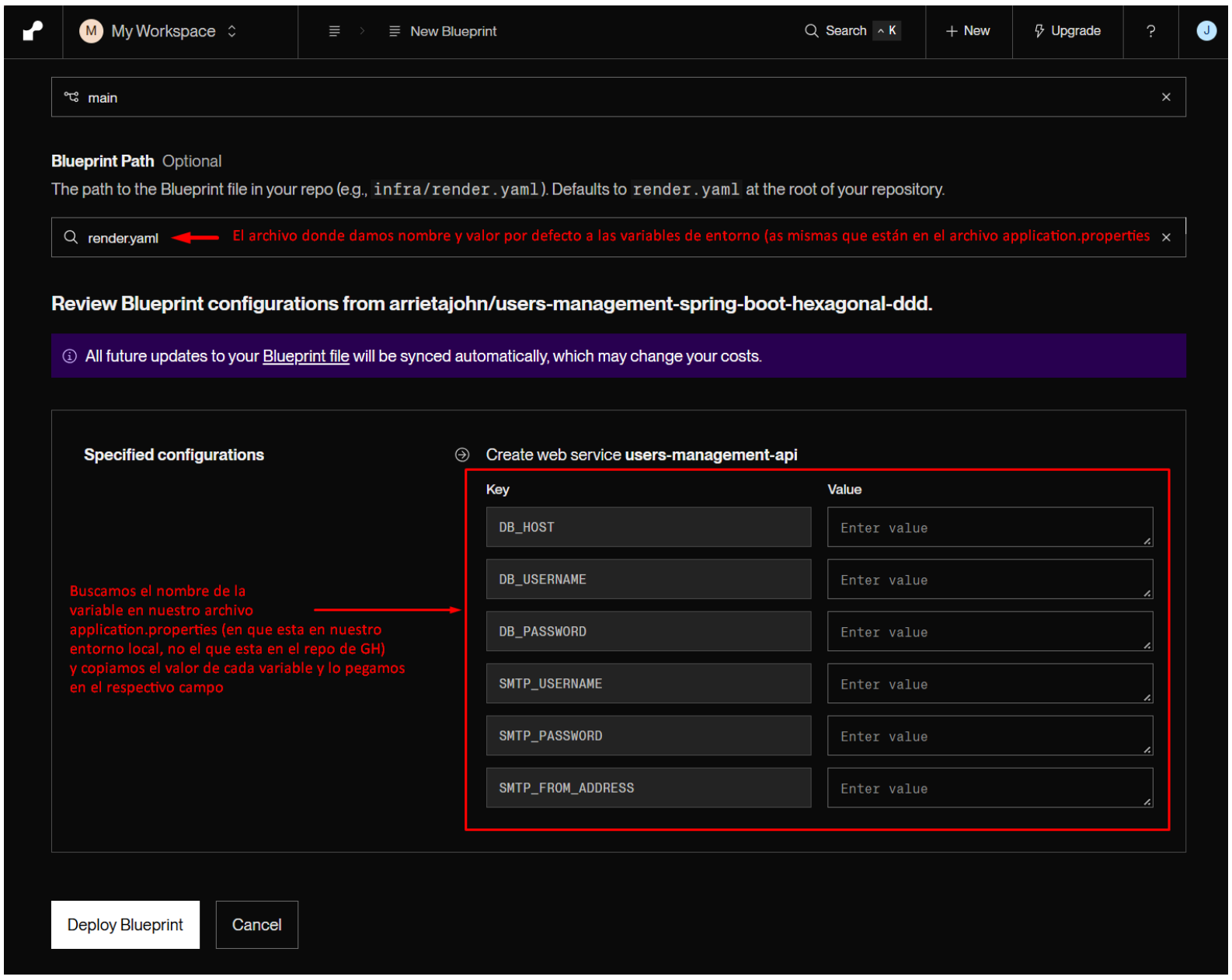


Figura 17: Detección del archivo render.yaml y campos para variables de entorno.

Paso 10: Detección y Configuración del Archivo Blueprint (render.yaml)

Render leerá el archivo de especificación de infraestructura como código (IaC) `render.yaml` presente en la raíz de tu repositorio. En esta vista se listan las variables de entorno declaradas que requieren un valor asignado antes del despliegue para inyectarse dinámicamente en el contenedor.

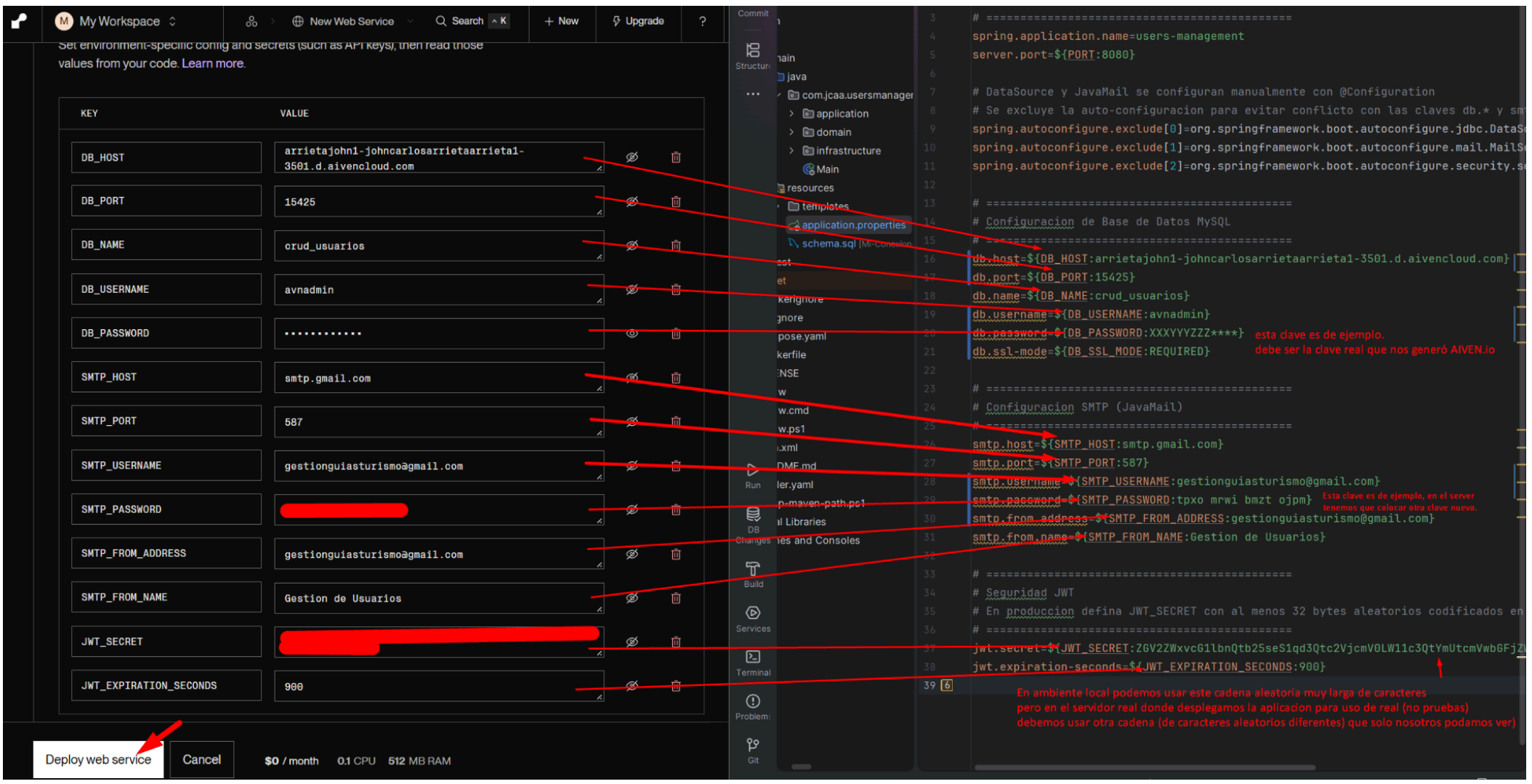


Figura 18: Asignación completa de las variables de entorno para la aplicación.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

Paso 11: Configuración Completa de Variables de Entorno

Copia y asigna las credenciales de producción para la Base de Datos (MySQL en Aiven.io), la configuración del servidor SMTP (Gmail) y el secreto para la firma de tokens JWT (JWT_SECRET). Como buena práctica de DevSecOps, nunca debes subir credenciales ni claves secretas directamente en el código fuente o repositorios de GitHub. Una vez configuradas todas las claves, haz clic en el botón **Deploy web service**.

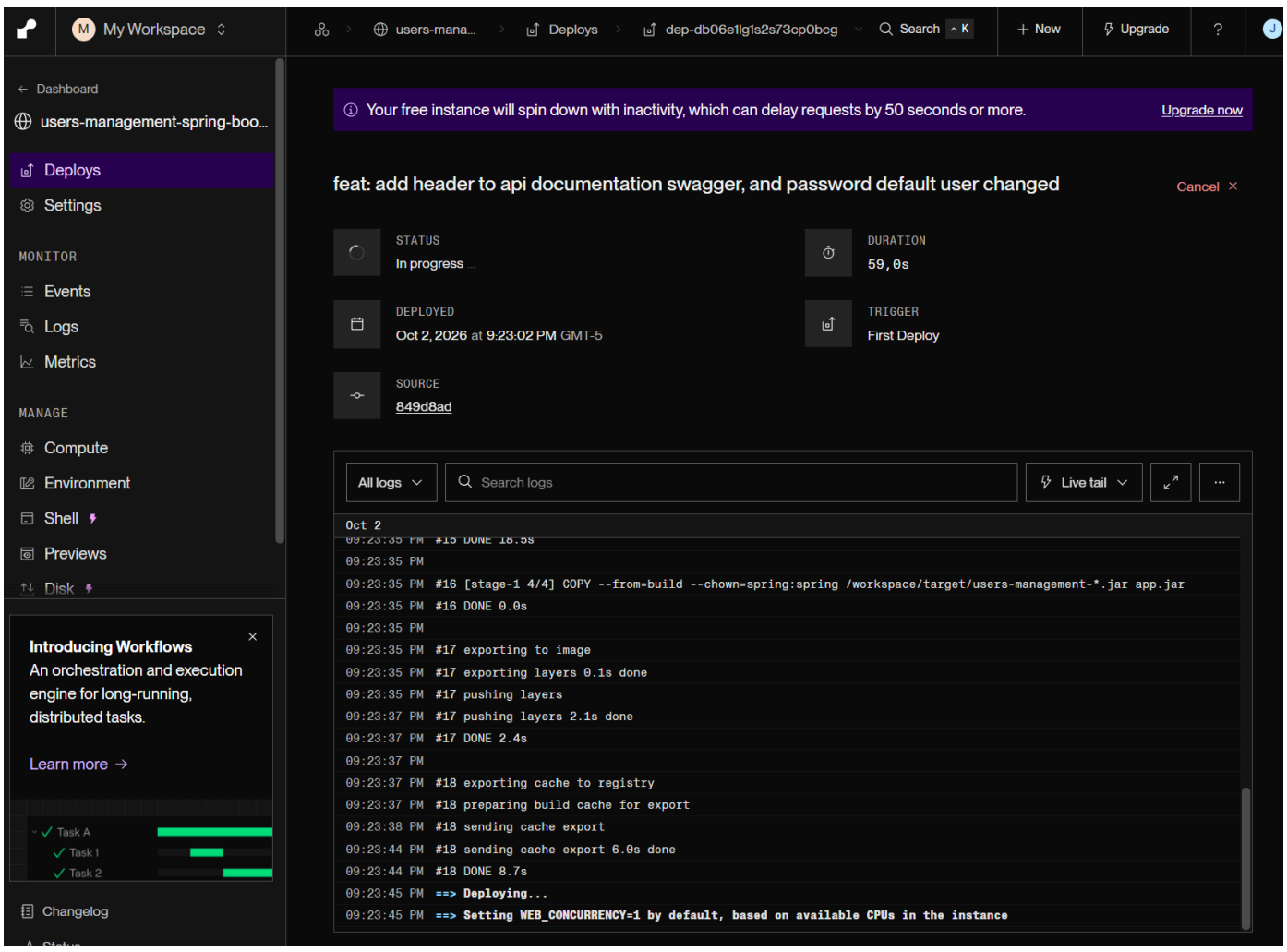


Figura 19: Consola de seguimiento de los logs del proceso de construcción y despliegue.

Paso 12: Monitoreo del Proceso de Construcción (Build & Deploy)

En la consola de **Deploys** puedes observar los logs en tiempo real mientras Render descarga el código, ejecuta el build de Docker (descargando dependencias Maven y compilando el JAR), empaqueta la aplicación Java y despliega el contenedor en el clúster.

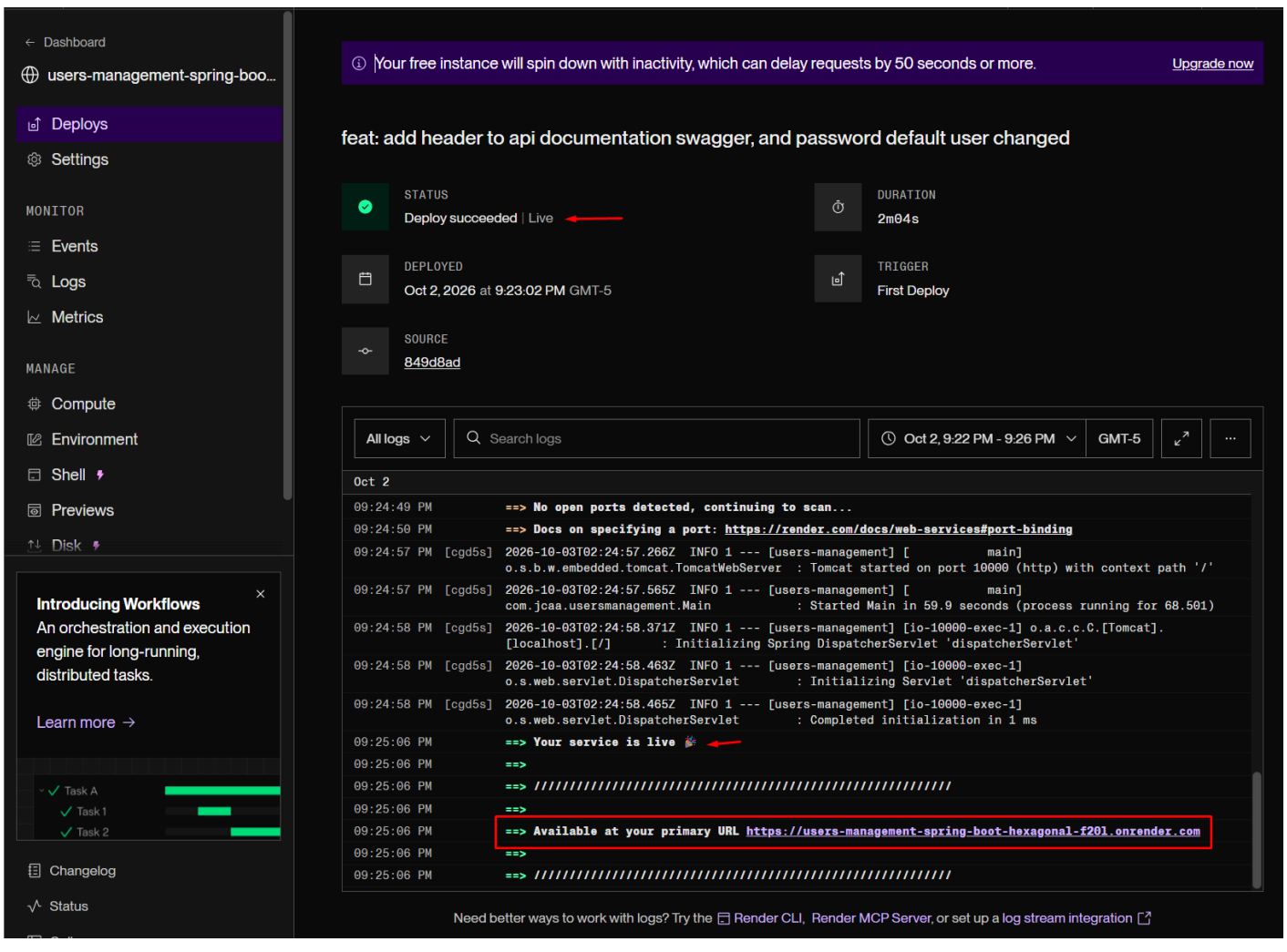


Figura 20: Despliegue exitoso y confirmación de la URL pública del servicio.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

Paso 13: Despliegue Exitoso y Obtención de la URL Pública

Cuando el estado cambie a **Deploy succeeded | Live**, la aplicación estará activa en la nube. En la parte inferior de los logs se indicará la URL pública HTTPS asignada a tu API (por ejemplo, `https://users-management-spring-boot-hexagonal-f20l.onrender.com`).

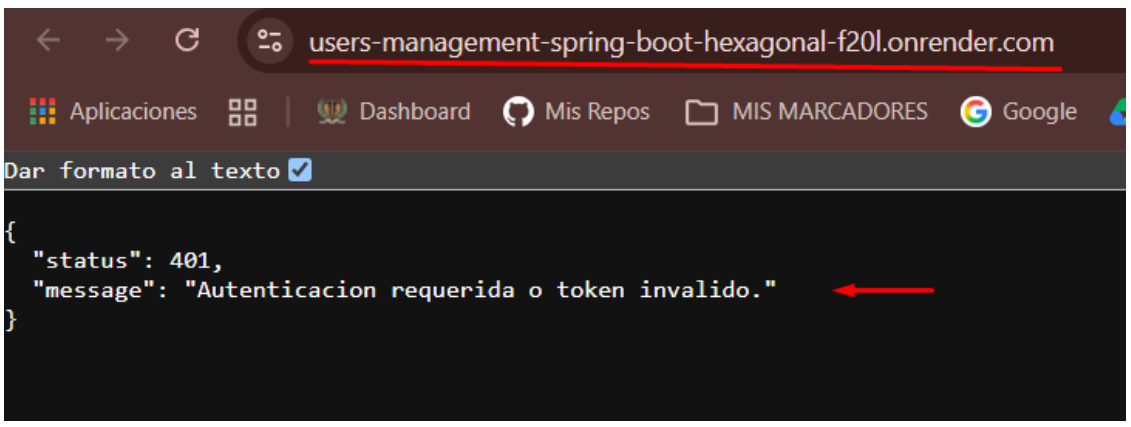


Figura 21: Respuesta HTTP 401 Unauthorized al acceder a la raíz de la API.

Paso 14: Verificación de Seguridad Inicial en la Raíz de la API

Al ingresar a la URL pública directamente desde el navegador, la aplicación retorna una respuesta JSON de estado HTTP **401 Unauthorized** (Autenticación requerida o token inválido). Técnicamente, este comportamiento confirma el correcto funcionamiento de la cadena de filtros de Spring Security (`FilterChain`), donde el filtro personalizado para la interceptación de JWT intercepta la petición entrante, constata la ausencia del encabezado `Authorization: Bearer <token>` y delega la gestión a la interfaz `AuthenticationEntryPoint` para rechazar el acceso de forma segura antes de alcanzar los controladores REST.

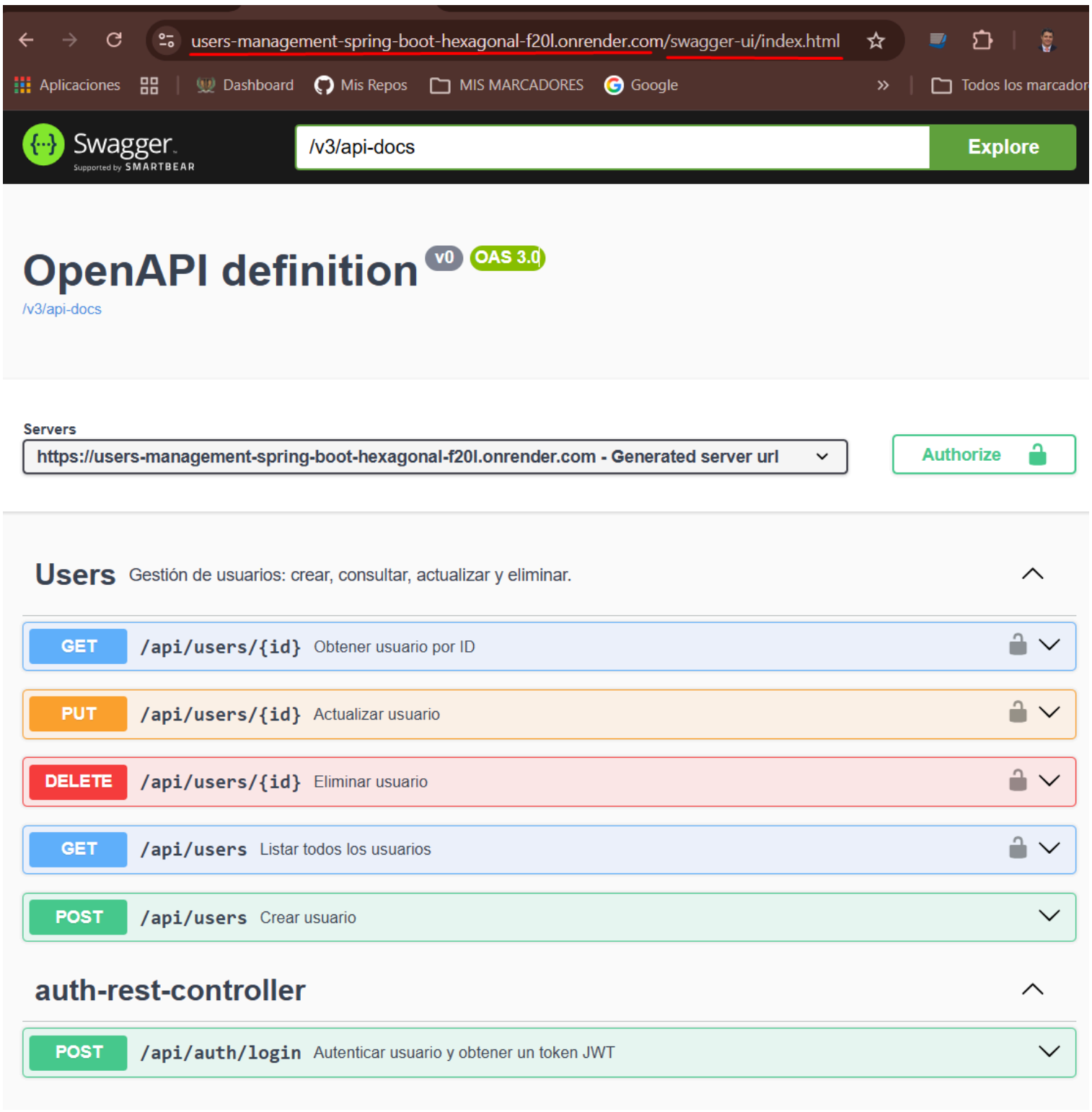


Figura 22: Interfaz de documentación interactiva Swagger UI de la API.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

Paso 15: Acceso a la Documentación Interactiva Swagger UI

Añade la ruta `/swagger-ui/index.html` al final de tu URL pública para ingresar a la interfaz interactiva OpenAPI (Swagger UI). Desde esta consola interactiva es posible inspeccionar la especificación técnica expuesta en formato OpenAPI 3.0, explorar los esquemas DTO de entrada/salida y validar visualmente los controladores REST habilitados. La infraestructura ha sido configurada explícitamente para permitir la consulta pública de los recursos de Swagger sin exigir token de sesión, garantizando que el diseño y contrato de la API sean completamente audibles.

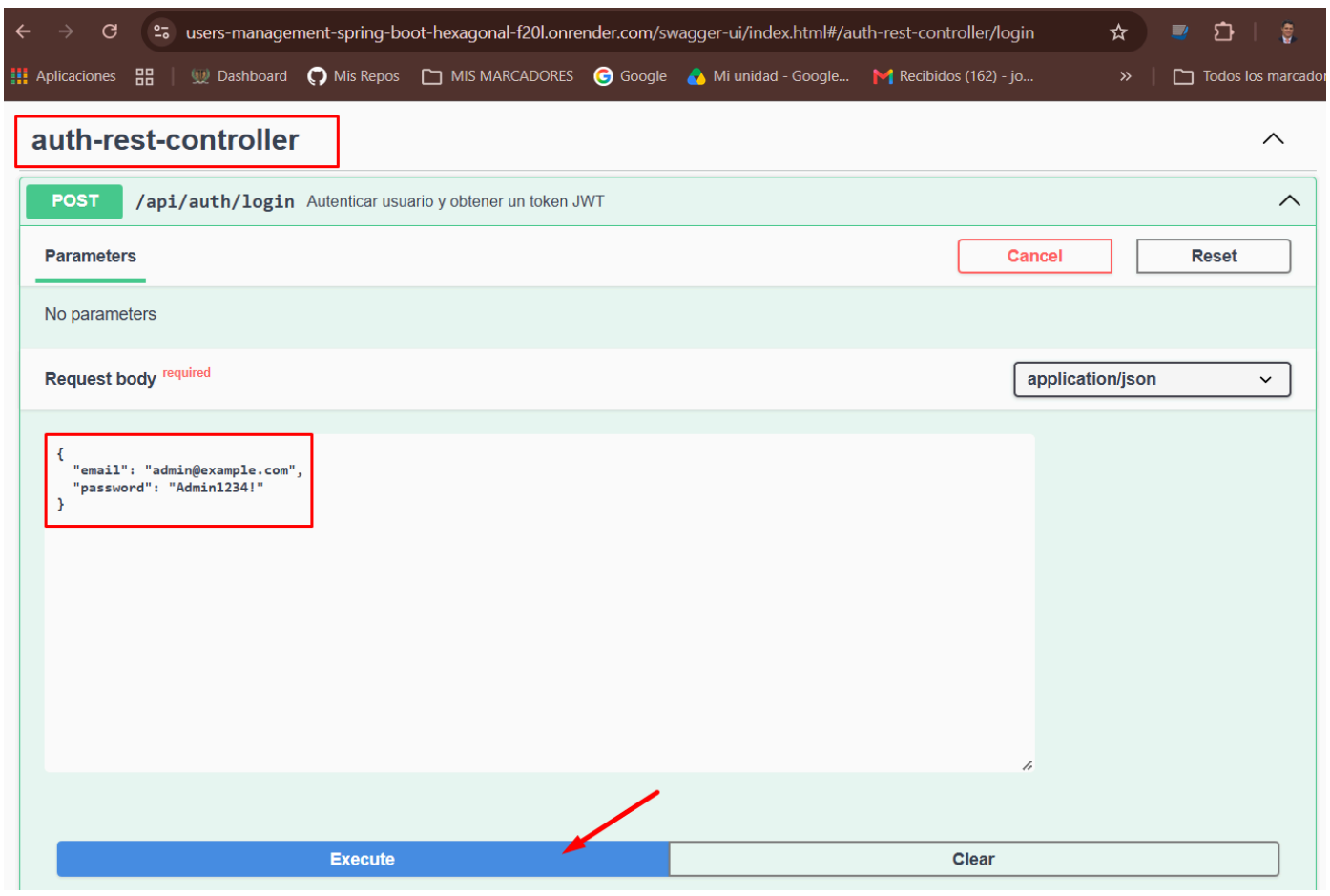


Figura 23: Petición de autenticación POST /api/auth/login en Swagger UI.

Paso 16: Inicio de Sesión Inicial vía Swagger UI

Despliega el endpoint **POST /api/auth/login**, haz clic en **Try it out** e ingresa las credenciales por defecto del usuario administrador en el cuerpo de la petición (JSON Payload) para autenticarte. El servicio procesa estas credenciales invocando internamente al gestor de autenticación (`AuthenticationManager`), el cual contrasta las claves con el hash almacenado en la base de datos (mediante BCrypt) y delega la emisión de un token firmado digitalmente al componente generador de JWT.

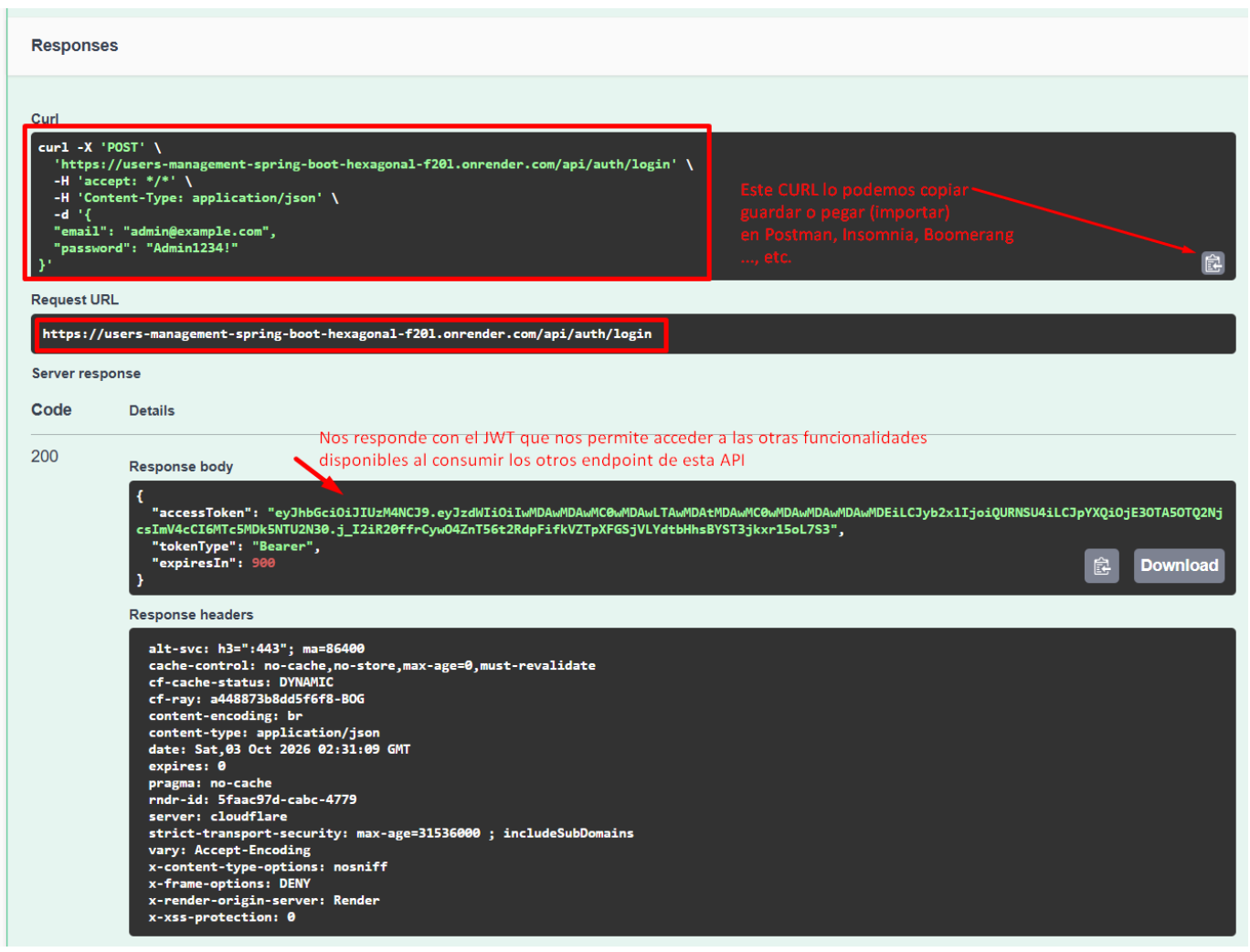


Figura 24: Respuesta con el token JWT e instrucción cURL generada.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

Paso 17: Obtención del Token JWT y Comando cURL

Al ejecutar la petición (código de respuesta **200 OK**), obtendrás el token de acceso (`accessToken`). Este JSON Web Token es una estructura autocontenida y sin estado (stateless) compuesta por encabezado (header), carga útil (payload) y firma HMAC SHA-256 codificada en Base64Url. Asimismo, Swagger genera automáticamente el comando **cURL** que encapsula las cabeceras, el tipo de contenido y la carga de datos, permitiendo importar fácilmente la solicitud hacia clientes HTTP externos como Postman, Insomnia o Boomerang para facilitar pruebas de integración automatizadas.

Como buena práctica de seguridad, lo primero que tenemos que hacer es cambiar la clave y el email del usuario admin registrados por default al crear la BD.
Esto lo podemos hacer usando el JWT que nos dan al iniciar sesión con las credenciales por default de este usuario, luego:

1. Creamos un REQUEST para el endpoint **GET /api/users** en Boomerang o Postman, o Insomnia, etc. usando los datos que nos muestra SWAGGER
    a. Usamos la URL del endpoint https://users-management-spring-boot-hexagonal-f20l.onrender.com/api/users
    b. Usamos el método HTTP GET
    c. Creamos una cabecera (HEADER) Authorization, le ponemos como valor Bearer El-JWT
    d. Enviamos la petición y esperamos los resultados

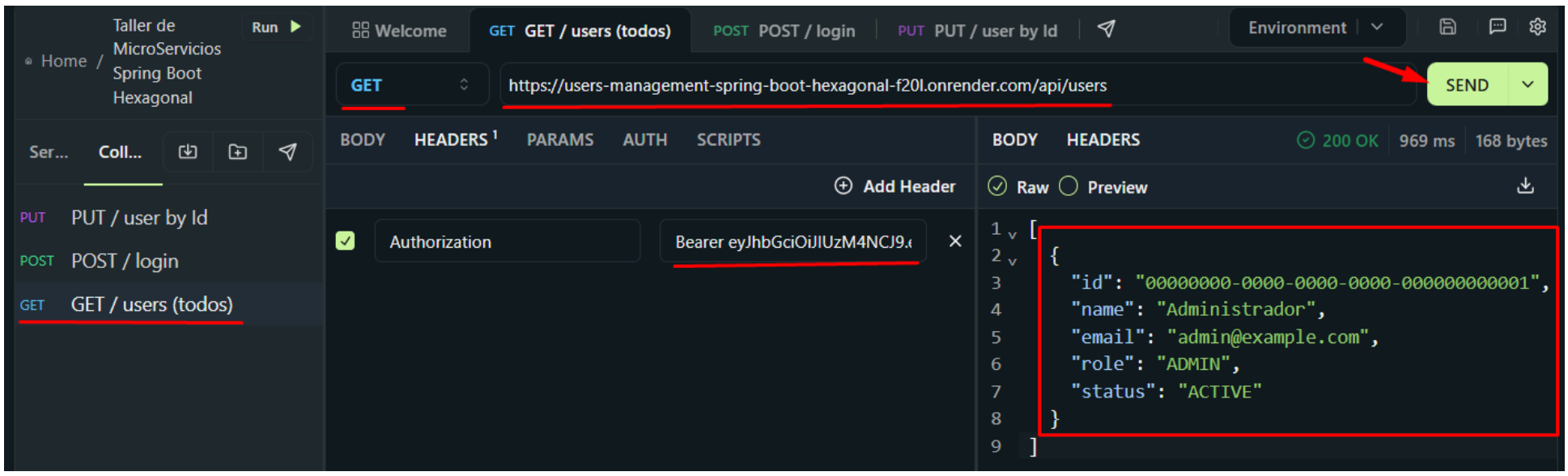


Figura 25: Consulta GET /api/users autorizada mediante JWT en un cliente HTTP.

2. Creamos un nuevo request para el endpoint PUT /api/users:
    a. usamos el ID de los datos que obtuvimos del usuario
    b. agregamos la cabecera Authorization con el Bearer JWT
    c. En el BODY (Payload) pegamos el JSON con los datos del usuario
    d. Modificamos su email y su password
    e. Enviamos la petición al servidor
    f. Esperamos la respuesta

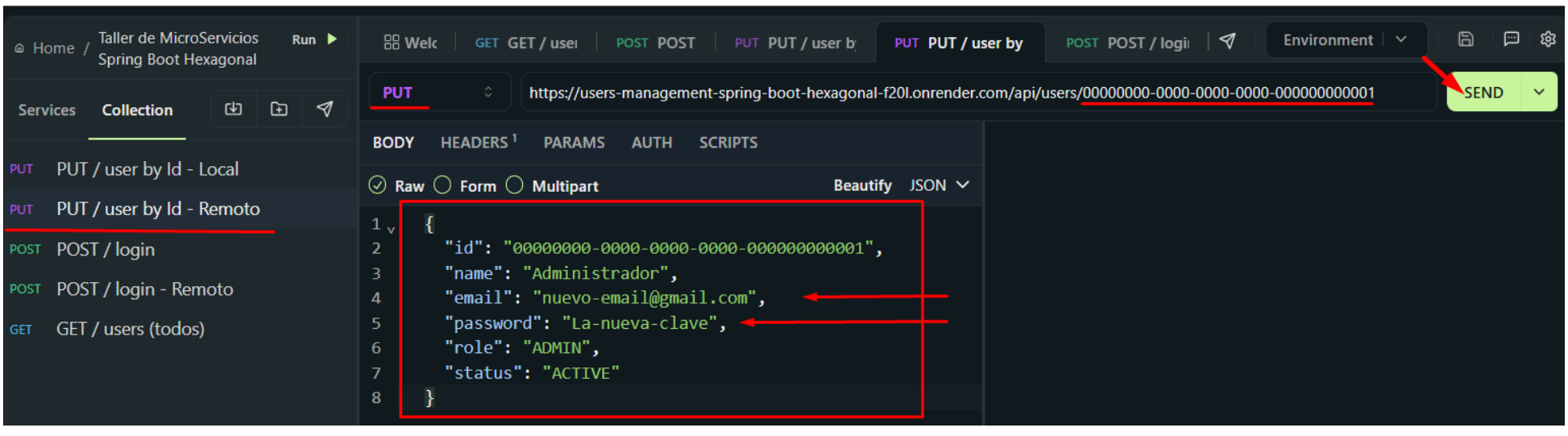


Figura 26: Actualización de datos del usuario mediante la petición PUT /api/users.

3. Creamos un nuevo Request POST /api/auth/login:
    a. Este endpoint no necesita autorización con JWT, porque este se usa justo para obtener el JWT
    b. En el BODY (Payload) escribimos el JSON con el nuevo email y la nueva contraseña del usuario
    c. Enviamos la petición al servidor
    d. Esperamos la respuesta

CC-BY.CC **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

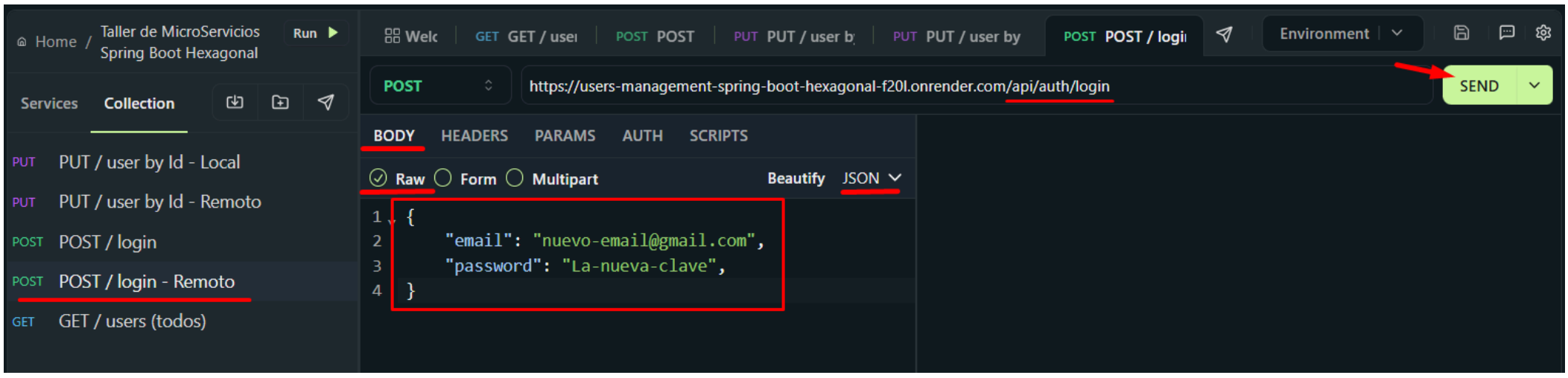


Figura 27: Inicio de sesión con las nuevas credenciales en el endpoint POST /api/auth/login.